

# HAI802I Algorithmique Avancée — TDs

Ivan Lejeune

19 avril 2026

## Table des matières

|     |                                                           |    |
|-----|-----------------------------------------------------------|----|
| 1   | Complexité . . . . .                                      | 3  |
| 1.1 | Quelques preuves en complexité des problèmes . . . . .    | 3  |
| 1.2 | Quelques problèmes dans les graphes . . . . .             | 4  |
| 1.3 | La classe de complexité $co\mathcal{NP}$ . . . . .        | 8  |
| 1.4 | Rappels en complexité . . . . .                           | 9  |
| 2   | Algorithmes d'approximation . . . . .                     | 10 |
| 2.1 | La classe <b>Log-APX</b> . . . . .                        | 10 |
| 2.2 | La classe <b>APX</b> . . . . .                            | 10 |
| 2.3 | Mesure différentielle . . . . .                           | 13 |
| 2.4 | Construction d'un FPTAS . . . . .                         | 14 |
| 2.5 | Polynomial-Time Asymptotic Scheme : PTAS . . . . .        | 14 |
| 2.6 | Points extrêmes . . . . .                                 | 14 |
| 2.7 | FPTAS et programmation dynamique . . . . .                | 15 |
| 2.8 | Schéma primal-dual . . . . .                              | 15 |
| 3   | Inapproximabilité ou seuil d'approximation . . . . .      | 16 |
| 3.1 | Seuil d'approximation . . . . .                           | 16 |
| 3.2 | Utilisation du principe Gap-réduction . . . . .           | 16 |
| 4   | Méthodes exactes . . . . .                                | 18 |
| 4.1 | Programmation dynamique . . . . .                         | 18 |
| 4.2 | Arbres de branchement . . . . .                           | 18 |
| 5   | Réductions divers . . . . .                               | 21 |
| 5.1 | $L$ -réduction . . . . .                                  | 21 |
| 6   | Quelques problèmes d'ordonnancement . . . . .             | 23 |
| 6.1 | Ordonnancement sur un processeur . . . . .                | 23 |
| 6.2 | Ordonnancement sur $m$ processeurs . . . . .              | 23 |
| 7   | Algorithmes randomisés . . . . .                          | 26 |
| 7.1 | Sur le problème MAXIMUM SATISFAISABILITÉ . . . . .        | 26 |
| 7.2 | Sur le problème COUVERTURE D'ENSEMBLES PONDÉRÉS . . . . . | 26 |
| 8   | Bornes inférieures et ETH . . . . .                       | 27 |

## Liste des Problèmes

|    |                                                   |    |
|----|---------------------------------------------------|----|
| 1  | 3-SAT . . . . .                                   | 3  |
| 2  | HALF 3-SAT . . . . .                              | 3  |
| 3  | $\neg$ 3-SAT . . . . .                            | 3  |
| 4  | NON EGAL SATISFAISABILITÉ (NAESAT) . . . . .      | 3  |
| 5  | EXACT COVER BY 3-SETS . . . . .                   | 4  |
| 6  | VERTEX COVER . . . . .                            | 4  |
| 7  | STEINER TREE . . . . .                            | 4  |
| 8  | COUPE MAXIMUM (CUT) . . . . .                     | 4  |
| 9  | ASYMMETRIC TSP PROBLEM (ATSP) . . . . .           | 17 |
| 10 | MAXIMUM $\neq$ 3-SAT . . . . .                    | 21 |
| 11 | MAXIMUM MULTI-COUPÉ (MAXIMUM MULTI-CUT) . . . . . | 21 |
| 12 | $P  \cdot  C_{\max}$ . . . . .                    | 23 |
| 13 | 2-PARTITION . . . . .                             | 23 |

# 1 Complexité

## 1.1 Quelques preuves en complexité des problèmes

### Exercice 1 Quelques preuves de NP-complétude autour de 3-SAT.

#### Problème 1 – 3-SAT

**Input:** Une formule  $\varphi(x_1, \dots, x_n)$  en 3-CNF

**Question:** Existe-t-il une affectation des variables  $(x_1, \dots, x_n)$  satisfaisant  $\varphi$  ?

#### Problème 2 – HALF 3-SAT

**Input:** Une formule  $\varphi(x_1, \dots, x_n)$  en 3-CNF

**Question:** Existe-t-il une affectation des variables  $(x_1, \dots, x_n)$  satisfaisant  $\varphi$  à 50% ?

#### Problème 3 – $\neg$ 3-SAT

**Input:** Une formule  $\varphi(x_1, \dots, x_n)$  en 3-CNF

**Question:** Existe-t-il une affectation des variables  $(x_1, \dots, x_n)$  qui ne satisfait pas  $\varphi$  ?

1. Montrer que  $\neg$ 3-SAT appartient à la classe  $P$ .
2. Montrer que HALF 3-SAT est NP-complet. Que dire du problème HALF 3-SAT dans le cas où le nombre de clauses satisfaites est au moins de 50% ?

### Solution.

1. On rappelle que tous les littéraux sont positifs, il suffit alors de tout mettre à faux pour trouver une solution, donc le problème appartient à la classe  $P$ .
2. On fait une réduction de 3-SAT vers HALF 3-SAT.  
On commence par faire la chose suivante :

$$I \rightsquigarrow I + \underbrace{(x \vee y \vee z)}_{m \text{ copies}}, \quad x, y, z \notin I$$

Si on a oui à 3-SAT alors on a  $m$  clauses satisfaites. On en déduit que sur  $2m$  clauses, il suffit de mettre  $x, y, z$  à faux et alors on a bien 50% des clauses qui sont satisfaites par la solution à 3-SAT.

Dans l'autre sens, si on a une solution à HALF 3-SAT on a plusieurs cas :

- Premier cas, on satisfait une clause dans  $I$ . Alors toutes les  $m$  clauses sont validées dans  $I$  et on a une solution à 3-SAT.
- Second cas, on ne satisfait aucune clause de  $I$ . Mais alors si dans  $I$  il n'y a aucune variable négative, on peut résoudre 3-SAT en mettant tout à positif, de même dans le cas où il n'y a aucune variable positive. Il y a donc forcément une variable présente de manière positive et négative dans  $I$ . Alors, au moins une clause est satisfaite. Donc on est dans le premier cas et forcément les  $m$  clauses satisfaites sont dans  $I$ .

Le problème de au moins 50% est facile. Il suffit de prendre une affectation aléatoire (ou son complémentaire).

### Exercice 2 Autour de Satisfaisabilité.

#### Problème 4 – NON EGAL SATISFAISABILITÉ (NAESAT)

**Input:** Une formule conjonctive  $\varphi$  sur  $n$  variables et  $m$  clauses

**Question:** Existe-t-il une affectation de valeurs de vérité aux variables qui satisfasse  $\varphi$  tel que chaque clause a un littéral vrai et un littéral faux ?

Montrer que NON EGAL SATISFAISABILITÉ est  $\mathcal{NP}$ -complet. La preuve se fera à partir de SATISFAISABILITÉ.

**Solution.** Le problème de NON EGAL SATISFAISABILITÉ est dans  $\mathcal{NP}$  car on peut vérifier en temps polynomial si deux formules booléennes sont non équivalentes. Pour montrer qu'il est  $\mathcal{NP}$ -complet, on effectue une réduction à partir du problème de SATISFAISABILITÉ. On montre que si on pouvait résoudre NON EGAL SATISFAISABILITÉ en temps polynomial, alors on pourrait résoudre SATISFAISABILITÉ en temps polynomial, ce qui contredit l'hypothèse que  $\mathcal{P} \neq \mathcal{NP}$ . Par conséquent, NON EGAL SATISFAISABILITÉ est  $\mathcal{NP}$ -complet.

Soit  $\varphi$  une formule conjonctive de 3-SAT. On construit une formule  $\psi$  pour le problème de NON EGAL SATISFAISABILITÉ de la manière suivante :

$$\psi_i = \varphi_i \wedge (x_1 \vee \dots \vee x_n) \wedge (\neg x_1 \vee \dots \vee \neg x_n), \quad \forall i \in \{1, \dots, m\}$$

Ensuite, on pose  $\psi = \bigwedge_{i=1}^m \psi_i$ .

Si  $\varphi$  est satisfaisable, alors il existe une affectation de valeurs de vérité aux variables qui satisfait  $\varphi$ . Cette même affectation satisfait également  $\psi$  car les clauses supplémentaires garantissent que chaque clause de  $\psi$  a un littéral vrai et un littéral faux.

Inversement, si  $\psi$  est satisfaisable, alors il existe une affectation de valeurs de vérité aux variables qui satisfait  $\psi$ . Cette affectation doit également satisfaire  $\varphi$  (car si elle satisfait chaque clause de  $\psi$ ), alors elle satisfait également chaque clause de  $\varphi$ .

Par conséquent,  $\varphi$  est satisfaisable si et seulement si  $\psi$  est satisfaisable, ce qui montre que le problème de NON EGAL SATISFAISABILITÉ est  $\mathcal{NP}$ -complet (car il est au moins aussi dur que SATISFAISABILITÉ, qui est lui-même  $\mathcal{NP}$ -complet).

## 1.2 Quelques problèmes dans les graphes

### Exercice 3 Quelques preuves de NP-complétude pour Steiner tree.

#### Problème 5 – EXACT COVER BY 3-SETS

**Input:** Un univers  $X$ , une collection  $C$  de triplés de  $X$

**Question:** Trouver une collection  $C' \subset C$  telle que chaque élément de  $X$  apparait exactement une fois dans les triplés de  $C'$

#### Problème 6 – VERTEX COVER

**Input:** Un graphe  $G = (V, E)$ , un entier  $k$

**Question:** Trouver  $V' \subset V$  telle que chaque arête est couverte par un sommet de  $V'$  avec  $|V'| = k$

#### Problème 7 – STEINER TREE

**Input:** Un graphe  $G = (V, E)$ , une fonction de poids  $w$  sur les arêtes, un ensemble de terminaux  $X \subset V$ , un entier  $k$

**Question:** Trouver  $T \subset E$  tel que la somme des poids des arêtes de  $T$  soit au plus de  $k$ ,  $G[T]$  est connexe et  $X \subset V(G[T])$

1. Montrer que STEINER TREE est NP-complet. Procéder à une réduction à partir de EXACT COVER BY 3-SETS.
2. Montrer que STEINER TREE est NP-complet par une réduction à partir de VERTEX COVER.

**Solution.**

### Exercice 4 Autour de Coupe maximum.

#### Problème 8 – COUPE MAXIMUM (CUT)

**Input:** Soit  $G = (V, E)$  un graphe non orienté et  $k \in \mathbb{N}$ .

**Question:** Existe-t-il une partition des sommets en deux sous-ensembles  $V_1$  et  $V_2$  tels que le nombre d'arêtes entre  $V_1$  et  $V_2$  soit  $k$  ?

Réduire NON EGAL 3-SAT à COUPE MAXIMUM. Conclure.

*Remarque :* vous verrez en master que coupe min est polynomiale même si les arêtes ont des poids.

**Solution.** On supposera que dans les clauses de taille 3 on a pas à la fois  $x$  et  $\bar{x}$ , sinon la clause est toujours satisfaite et on peut l'ignorer. On suppose de plus que toute paire de clauses a au plus un littéral en commun. On procède alors à la construction suivante :

**Construction 1.** On crée un graphe  $G$  qui, à chaque littéral, crée un sommet  $v$  et un sommet  $\bar{v}$ .

On obtient alors que le nombre d'arêtes sur ce graphe est égal à 3 fois le nombre de clauses de  $\varphi$  plus une par variable, soit  $3m + n$ . De plus, on a

$$\sum_{x_i \in L} n_{x_i} = 3m$$

Alors, on cherche une coupe telle que

$$C(S, \bar{S}) \geq 2m + n$$

Soit  $t$  une solution de NON EGAL 3-SAT. Supposons que  $n$  variables sont mises à vrai, et donc  $n$  variables sont mises à faux. Alors, on a  $2m$  arêtes de clauses coupées et  $n$  arêtes de variables coupées, soit un total de  $2m + n$  arêtes coupées. Plus précisément, avec les  $2m$  arêtes restantes, on ne peut couper que des triangles. Ces triangles, ayant au moins 1 sommet dans chaque partie de la coupe, sont forcément coupés 2 fois. Il y en a  $m$  au total, soit  $2m$  arêtes coupées.

**Exercice 5 title.**

**Solution.**

**Exercice 6 title.**

**Solution.**

**Exercice 7 title.**

**Solution.**

**Exercice 8 Problème de la coloration.**

1. Montrer que 3-COLORATION est  $\mathcal{NP}$ -complet en effectuant une réduction à partir de 3-SAT.

Nous proposons la transformation polynomiale suivante à partir de 3-SAT :

Soit  $I$  une instance de 3-SAT constitué d'un ensemble  $B = \{C_1, \dots, C_m\}$  de clauses sur

un ensemble  $X = \{x_1, \dots, x_n\}$  de variables. Nous allons construire, à partir de la formule  $B$ , un graphe  $G = (S, A)$  de manière que  $B$  soit satisfaisable si et seulement si  $G$  admet une coloration en 3 couleurs.

- Pour chaque variable  $x_i$ , on pose  $T_i = (S_{i,1}, A_{i,1})$  où  $S_{i,1} = \{x_i, \bar{x}_i\}$  et  $A_{i,1} = \{(x_i, \bar{x}_i)\}$ , voir figure 1.
- Pour chaque clause  $C_j$ , on pose  $V_j = (S_{j,2}, A_{j,2})$  où  $S_{j,2} = \{y_{j,1}, y_{j,2}, y_{j,3}, y_{j,4}, y_{j,5}, y_{j,6}\}$  et  $A_{j,2} = \{(y_{j,1}y_{j,2}), (y_{j,1}y_{j,4}), (y_{j,2}y_{j,4}), (y_{j,4}y_{j,5}), (y_{j,5}y_{j,6}), (y_{j,5}y_{j,3}), (y_{j,3}y_{j,6})\}$ , voir figure 2.
- $V_j$  est isomorphe à  $G_0 - \{u_0, u_1, u_2\}$ .
- On pose  $T = (S_3, A_3)$  où  $S_3 = \{v_1, v_2\}$  et  $A_3 = \{(v_1v_2)\}$ .
- Pour chaque variable  $x_i$  on pose  $A_{i,4} = \{v_2x_i, v_2\bar{x}_i\}$  et  $A_{i,5} = \{(v_1y_{j,6}, v_2y_{j,6})\}$  pour tout  $j \in \{1, \dots, m\}$ .
- Enfin,  $A_{j,6}$  est un ensemble de trois arêtes reliant, pour chaque clause  $C_j$ , les trois sommets de type  $x_i$  ou  $\bar{x}_i$  correspondant aux littéraux de  $C_j$ . Le premier littéral de  $C_j$  est relié à  $y_{j,1}$ , le deuxième à  $y_{j,2}$  et le troisième à  $y_{j,3}$ , voir figure 3.

Tous les graphes présentés ci-dessus sont donnés dans les figures ci-dessous :

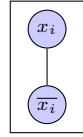


FIGURE 1

—  
Graphe  
 $T_i$

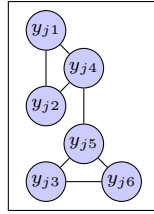


FIGURE 2 –  
Graphe  $V_j$

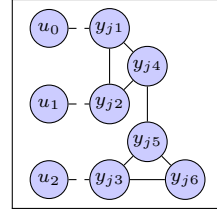


FIGURE 3 – Graphe  
 $G_0$

Donner le nombre de sommets et le nombre d'arêtes de  $G$ . Appliquer la construction sur l'instance  $X = \{x_1, x_2, x_3, x_4, x_5\}$  et  $B = \{C_1, C_2\}$  où  $C_1 = (x_1 \vee \bar{x}_3 \vee \bar{x}_4)$  et  $C_2 = (\bar{x}_4 \vee x_2 \vee \bar{x}_1)$ .

2. Montrer que le problème de 4-coloration est  $\mathcal{NP}$ -complet en effectuant une réduction à partir de 3-COLORATION.
3. Regarder la preuve pour des graphes planaires.

**Solution.** On propose de visualiser le schéma de construction avec la figure 4 suivante :

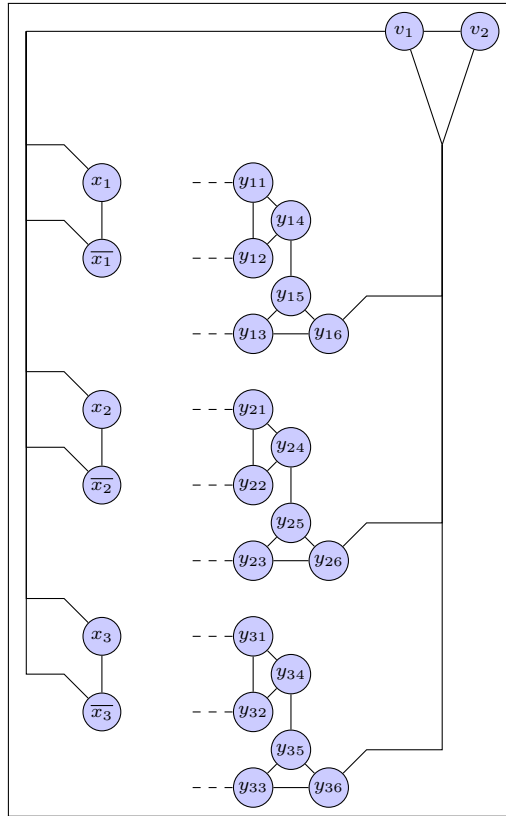


FIGURE 4 – Construction de la réduction de 3-COLORATION à partir de 3-SAT

En appliquant le schéma de construction à l'instance donnée, on obtient le graphe suivant :

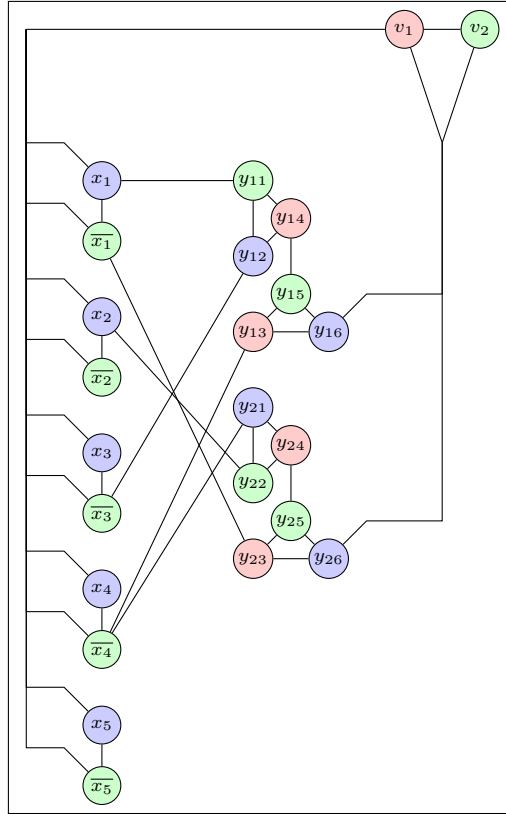


FIGURE 5 – Application de la construction à l'instance donnée avec un exemple de coloration

On peut aussi retirer les couleurs de ce graphe et essayer de les assigner manuellement. Les paires  $(x_i, \bar{x}_i)$  doivent être colorées avec des couleurs différentes, et  $v_1$  et  $v_2$  doivent être colorés avec des couleurs différentes. Les  $y_{j,6}$  doivent être colorés différemment de  $v_1$  et  $v_2$ . Avec ces seules contraintes, on peut trouver une coloration en 3 couleurs du graphe, ce qui correspond à une assignation de vérité pour les variables  $x_i$  qui satisfait les clauses  $C_j$  et on peut passer de l'une à l'autre de manière bijective.

Les seules choses à considérer pour passer d'une coloration à une autre est la permutation des couleurs  $y_{j,1}, y_{j,2}$  et celles des  $x_i$  et  $\bar{x}_i$ , l'extension de cette construction à des instances plus grandes est alors facile.

1. Il est facile de voir qu'avec la construction proposée on a bien une réduction de 3-SAT vers 3-COLORATION et donc ce dernier est  $\mathcal{NP}$ -complet.
2. Il suffit de considérer le graphe  $G'$  qui a les memes sommets et arêtes que  $G$  mais avec un sommet supplémentaire  $v$  connecté à tous les autres sommets de  $G$ . Alors si  $G$  est 3-colorable, alors  $G'$  est 4-colorable (en coloriant  $v$  avec une nouvelle couleur). Inversement, si  $G'$  est 4-colorable, alors  $v$  doit être colorié avec une couleur différente de celle de tous les autres sommets de  $G$ , ce qui implique que  $G$  est 3-colorable.

### 1.3 La classe de complexité $\mathcal{coNP}$

**Exercice 9 title.**

**Solution.**



Exercice 10 title.

Solution.

Exercice 11 title.

Solution.

## 1.4 Rappels en complexité

Exercice 12 title.

Solution.

## 2 Algorithmes d'approximation

### 2.1 La classe Log-APX

Exercice 13 title.

Solution.

### 2.2 La classe APX

Exercice 14 title.

Solution.

Exercice 15 title.

Solution.

Exercice 16 title.

Solution.

Exercice 17 title.

Solution.

**Exercice 18 Vertex Cover dans les planaires utilisant une 4-coloration.**

1. Rappeler le résultat de complexité pour le problème de l'étape 4-coloration.
2. Expliquer le fonctionnement de l'algorithme 6.
3. Prouver la correction de l'algorithme 6.
4. Montrer que le problème admet un ratio de  $\frac{3}{2}$ .

**Solution.**

1. Le problème de la 4-coloration dans un graphe planaire est polynomial.
2. x
3. La couleur retenue est au moins de taille  $\frac{w}{4}$  où  $w$  est

$$w = \sum_{v \in V} w(v)$$

On conserve les sommets de toutes les autres couleurs, ils couvrent donc les sommets de la couleur retenue. Comme ces sommets forment un stable, on couvre bien toutes les arêtes du graphe.

4. On a

$$OPT_{LP} = \sum_{\substack{v \in V \\ x_v^* = 1}} x_v c(v) + \sum_{\substack{v \in V \\ x_v^* = \frac{1}{2}}} x_v c(v)$$

Il en suit

$$\begin{aligned} S &= OPT_{LP} + \frac{1}{2} \text{Cost}(\bar{k}) - \frac{1}{2} \text{Cost}(k) \\ &\leq OPT_{LP} + \frac{w}{4} \\ &\leq OPT_{LP} + \frac{1}{2} OPT_{LP} \\ &\leq \frac{3}{2} OPT_{LP} \\ &\leq \frac{3}{2} OPT_{ILP} \\ &\Rightarrow \rho \leq \frac{3}{2} \end{aligned}$$

**Exercice 19 title.**

**Solution.**

**Exercice 20 title.**

**Solution.**

**Exercice 21 Approximation pour Maximum Independent : algorithme glouton.**

1. Donner la complexité de l'algorithme et montrer que la solution donnée par l'algorithme est un stable.
2. A partir de l'algorithme, proposer un graphe qui donne le pire cas.
3. Soit  $G = (V, E)$  un graphe ayant  $n$  sommets et  $m$  arêtes, soit  $\delta = \frac{m}{n}$  la densité du graphe de  $G$ . Montrer que toute solution  $S$  est d'au moins  $\frac{n}{2\delta+1}$ .
4. Montrer maintenant que le ratio est  $\delta + 1$ .

**Solution.**

- 1.
2. On propose le graphe suivant Où  $A = K_n, B = IS_n, C = \{x\}$  où  $C$  domine  $B$  et  $A$  et  $B$  sont complètement connectés. L'algorithme peut choisir  $x$  et ne pas choisir les sommets de  $B$  alors que la solution optimale est de choisir tous les sommets de  $B$ . Ainsi on a bien un ratio de  $\frac{n}{2}$  qui peut être arbitrairement grand.
3. Sachant que l'algorithme s'arrête, lorsqu'on a plus de sommets à traiter, on a :

$$\sum_{i \in S} (d_i + 1) = n \tag{1}$$

où  $d_i$  est le degré du sommet  $i$  dans le graphe restant.

De plus, on a

$$\sum_{i \in S} \frac{d_i(d_i + 1)}{2} \leq m \quad (2)$$

En combinant les équations (1) et (2), on a

$$\begin{aligned} \sum_{i \in S} (d_i + 1) \cdot \sum_{i \in S} d_i(d_i + 1) &\leq 2m + n \\ &\leq 2\delta m + n - n(2S + 1) \end{aligned}$$

En appliquant l'inégalité de Cauchy-Schwarz avec  $a_k = (d_i + 1)$  et  $b_k = 1$ , on a

$$\begin{aligned} \left( \sum_{i \in S} (d_i + 1) \cdot 1 \right)^2 &\leq \left( \sum_{i \in S} (d_i + 1)^2 \right) \cdot \left( \sum_{i \in S} 1^2 \right) \\ &\leq \left( \sum_{i \in S} (d_i + 1)^2 \right) \end{aligned}$$

et on a bien

$$\frac{n}{2\delta + 1} \leq |S|$$

**Exercice 22 title.**

**Solution.**

**Exercice 23 title.**

**Solution.**

**Exercice 24 title.**

**Solution.**

**Exercice 25 Algorithme glouton pour SAT.** Montrer que l'algorithme admet une performance relative de  $\frac{1}{2}$ . On pourra utiliser une preuve par réduction.

**Solution.** On fait une preuve par récurrence.

Dans le cas de base, le résultat est trivial.

Supposons maintenant qu'on a  $n$  variables. Soit  $v$  la variable dans  $\varphi$  qui apparaît le plus. Soit  $c_1$  le nombre de clauses où  $v$  apparaît positivement et  $c_2$  le nombre de clauses où  $v$  apparaît négativement. Sans perte de généralité, supposons que  $c_1 \geq c_2$ . En assignant  $v$  à vrai, on satisfait au moins  $c_1$  clauses, il reste alors  $c - c_1$  clauses à satisfaire sur les  $n - 1$  variables restantes, où  $c$  est le nombre de clauses satisfaites par une affectation optimale. Par hypothèse de récurrence, l'algorithme satisfait au moins  $\frac{1}{2}(c - c_1)$  clauses parmi les  $n - 1$  variables restantes. Le nombre

de clauses satisfaites au total par l'algorithme est alors au moins

$$c_1 + \frac{1}{2}(c - c_1) = \frac{1}{2}c + \frac{1}{2}c_1 \geq \frac{1}{2}c.$$

**Exercice 26 title.**

**Solution.**

**Exercice 27 title.**

**Solution.**

**Exercice 28 title.**

**Solution.**

**Exercice 29 title.**

**Solution.**

## 2.3 Mesure différentielle

**Exercice 30 Propriétés liées à la mesure au pire cas.**

1. Soit  $\Pi$  un problème de maximisation appartenant à la classe  $\mathcal{NPO}$  ayant un ratio  $\delta$  pour la mesure. Montrer que cela implique que le ratio dans le pire cas est de  $\delta$ .
2. Sur le problème du TSP, considérons une instance  $I = (K_n, \vec{d})$  avec  $\vec{d}$  le vecteur des arêtes-distances. Alors, pour n'importe quelle transformation affine

$$\vec{d} := \gamma \cdot \vec{d} + \mu \cdot \vec{1}$$

avec  $\gamma, \mu \in \mathbb{Q}$  produit une approximation différentielle équivalente à celle obtenue pour  $I$ .

**Solution.**

**Exercice 31 Mesure différentielle : borne inférieure pour le problème du Bin Packing.**

Montrer que BIN PACKING ne peut être résolu par un algorithme polynomial à rapport différentiel minoré par  $\frac{n-1}{n}$ .

Pour cela, on pourra supposer l'existence d'un tel algorithme et arriver à une contradiction.

**Solution.** On suppose qu'il existe un algorithme polynomial  $A$  pour BIN PACKING tel que

$$\frac{n - A(I)}{n - \text{OPT}(I)} \geq \frac{n - 1}{n}$$

alors

$$A(I) - \text{OPT}(I) \leq 1 - \frac{\text{OPT}(I)}{n} < 1$$

or  $A(I)$  et  $\text{OPT}(I)$  sont des entiers, donc  $A(I) = \text{OPT}(I)$  et  $A$  est un algorithme polynomial pour BIN PACKING, ce qui est impossible.

**Exercice 32 title.**

**Solution.**

## 2.4 Construction d'un FPTAS

**Exercice 33 title.**

**Solution.**

**Exercice 34 title.**

**Solution.**

## 2.5 Polynomial-Time Asymptotic Scheme : PTAS

**Exercice 35 title.**

**Solution.**

**Exercice 36 title.**

**Solution.**

## 2.6 Points extrêmes

**Exercice 37 Solutions semi-intégrales pour le problème de vertex cover.**

Par définition, une solution réalisable  $x$  est appelée semi-intégrale si les valeurs appartiennent à l'ensemble  $\{0, \frac{1}{2}, 1\}$ .

1. Donner la formulation du problème de VERTEX COVER par un programme linéaire en nombres entiers

**Solution.**

**Exercice 38 title.**

**Solution.**

**Exercice 39 title.**

**Solution.**

## 2.7 FPTAS et programmation dynamique

**Exercice 40 title.**

**Solution.**

On peut trouver et même construire la solution avec le tableau suivant :

|           | $T(i, j)$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
|-----------|-----------|---|---|---|---|---|---|---|---|---|---|----|----|----|----|----|----|----|
| $a_1 = 5$ | 1         | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0  | 0  | 0  | 0  | 0  | 0  | 0  |
| $a_2 = 9$ | 2         | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 1 | 0  | 0  | 0  | 0  | 1  | 0  | 0  |
| $a_3 = 3$ | 3         | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 1 | 0  | 0  | 1  | 0  | 1  | 0  | 0  |
| $a_4 = 8$ | 4         | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 | 1 | 0  | 1  | 1  | 1  | 1  | 0  | 1  |
| $a_5 = 2$ | 5         | 0 | 0 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1  | 1  | 1  | 1  | 1  | 1  | 1  |
| $a_6 = 5$ | 6         | 0 | 0 | 1 | 1 | 0 | 1 | 0 | 1 | 1 | 1 | 1  | 1  | 1  | 1  | 1  | 1  | 1  |

Pour le problème du sac à dos, on peut reprendre le même tableau mais avec les nouvelles données, ce qui donne :

|                        | $K(j, w)$ | 0 | 1 | 2 | 3 | 4 | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 |
|------------------------|-----------|---|---|---|---|---|----|----|----|----|----|----|----|----|
| $(w_1, v_1) = (1, 1)$  | 1         | 0 | 1 | 1 | 1 | 1 | 1  | 1  | 1  | 1  | 1  | 1  | 1  | 1  |
| $(w_2, v_2) = (2, 6)$  | 2         | 0 | 1 | 6 | 7 | 7 | 7  | 7  | 7  | 7  | 7  | 7  | 7  | 7  |
| $(w_3, v_3) = (5, 18)$ | 3         | 0 | 1 | 6 | 7 | 7 | 18 | 19 | 24 | 25 | 25 | 25 | 25 | 25 |
| $(w_4, v_4) = (6, 22)$ | 4         | 0 | 1 | 6 | 7 | 7 | 18 | 22 | 24 | 28 | 29 | 29 | 40 | 41 |
| $(w_5, v_5) = (7, 24)$ | 5         | 0 | 1 | 6 | 7 | 7 | 18 | 22 | 24 | 28 | 30 | 31 | 40 | 42 |

## 2.8 Schéma primal-dual

**Exercice 41 title.**

**Solution.**

## 3 Inapproximabilité ou seuil d'approximation

### 3.1 Seuil d'approximation

**Exercice 42 title.**

**Solution.**

**Exercice 43 title.**

**Solution.**

**Exercice 44 title.**

**Solution.**

### 3.2 Utilisation du principe Gap-réduction

**Exercice 45 Non-approximation du TSP en fonction de paramètres.**

Nous voulons montrer que si  $\mathcal{P} \neq \mathcal{NP}$ , alors aucun algorithme polynomial pour le problème de la minimisation du VOYAGEUR DE COMMERCE ne peut garantir un rapport d'approximation classique inférieur à  $\frac{d_{\max}}{nd_{\min}}$ , où  $d_{\max}$  et  $d_{\min}$  sont respectivement la plus grande et la plus petite des distances sur les arêtes de l'instance du TSP, et  $n$  est le nombre de villes.

Pour cela, procéder par l'absurde et utiliser le même principe qu'en cours. A partir d'un graphe quelconque  $G = (V, E)$  :

1. Construire un graphe complet valué en complétant les arêtes.
2. Executer votre algorithme sur votre instance et conclure sur l'existence d'un cycle Hamiltonien dans  $G$ .

**Solution.**

1. On considère le graphe  $G' = (V', E')$  suivant :  $V' = V$  et  $E' =$  l'ensemble de toutes les paires de sommets de  $V$ . On définit la fonction de coût  $c$  sur les arêtes de  $G'$  de la manière suivante :

$$c(u, v) = \begin{cases} 1 & \text{si } (u, v) \in E, \\ d_{\max} & \text{sinon.} \end{cases}$$

Par simplification, on peut supposer que  $d_{\min} = 1$  et on peut poser  $d_{\max} = M$ .

2. Supposons que nous avons un algorithme polynomial  $A$  pour le TSP qui garantit un rapport d'approximation inférieur à  $\frac{M}{n}$ . Si  $G$  contient un cycle Hamiltonien, alors le coût optimal du TSP sur  $G'$  est  $n$  (car on peut parcourir les arêtes de coût 1). Par conséquent, l'algorithme  $A$  doit trouver une solution de coût au plus  $\frac{M}{n} \cdot n = M$ .

Si  $G$  ne contient pas de cycle Hamiltonien, alors le coût optimal du TSP sur  $G'$  est au moins  $n - 1 + M$  (car on doit utiliser au moins une arête de coût  $M$ ). Par conséquent, l'algorithme  $A$  doit trouver une solution de coût au plus  $\frac{M}{n} \cdot (n - 1 + M)$  qui est strictement supérieur à  $M$  pour  $n > 1$ .

On a donc séparé les cas où  $G$  contient un cycle Hamiltonien de ceux où il n'en contient pas et on a trouvé un écart de coût, il est donc impossible d'approcher une solution du TSP



avec un rapport d'approximation inférieur à  $\frac{M}{n}$ , ce qui contredit l'existence de l'algorithme  $A$ . Par conséquent, si  $\mathcal{P} \neq \mathcal{NP}$ , aucun algorithme polynomial pour le TSP ne peut garantir un rapport d'approximation inférieur à  $\frac{d_{\max}}{nd_{\min}}$ .

#### Exercice 46 TSP asymétrique.

On rappelle que le problème du TSP asymétrique s'énonce comme suit :

##### Problème 9 – ASYMMETRIC TSP PROBLEM (ATSP)

**Input:** Un graphe  $G = (V, E)$  orienté et muni d'une fonction de coût positive sur les arcs.

**Question:** Trouver un circuit de coût minimum qui passe par chaque sommet exactement une fois.

Montrer que ATSP est  $\mathcal{NP}$ -complet et non approximable.

**Solution.** Le problème du ATSP est un cas particulier du TSP (en imposant des coûts infinis pour les arcs non présents), et que le TSP est déjà connu pour être  $\mathcal{NP}$ -complet et non approximable. Par conséquent, ATSP hérite de ces propriétés.

#### Exercice 47 title.

**Solution.**

#### Exercice 48 title.

**Solution.**

#### Exercice 49 title.

**Solution.**

## 4 Méthodes exactes

### 4.1 Programmation dynamique

Exercice 50 title.

Solution.

Exercice 51 title.

Solution.

### 4.2 Arbres de branchement

Exercice 52 title.

Solution.

**Exercice 53 Complexité paramétrique : Kernelization pour le problème de Vertex Cover.**

**Définition 1.** Un problème  $\Pi$  de paramètre  $k$  est de **complexité paramétrique polynomiale** (FPT) si la complexité en temps de  $\Pi$  pour une instance de taille  $n$  est bornée par  $f(k) \cdot n^{O(1)}$  pour une certaine fonction  $f$ .

Dans la suite nous allons nous intéresser au problème de VERTEX COVER avec pour paramètre  $k$ .

Nous allons procéder à la kernelization du problème de VERTEX COVER. Pour finir, nous proposerons une méthode exacte, utilisant en temps exponentiel  $k$  pour résoudre de manière exacte le problème. Pour cela nous allons procéder à plusieurs réductions de l'instance initiale et après utiliser un arbre borné de recherche.

- Si  $x$  est un sommet isolé alors  $x$  n'appartient à aucun vertex cover de  $G$  et on peut le supprimer de  $G$ .
- Si  $x$  est un sommet de degré 1 voisin de  $y$  alors il existe une solution optimale contenant  $y$  donc

$$VC(G, k) = VC(G \setminus \{x, y\}, k - 1) \cup \{y\}$$

- Si  $x$  est un sommet de degré  $\geq k + 1$ , alors si  $G$  possède une solution de taille  $k$  alors  $x$  doit appartenir à cette solution. Donc

$$VC(G, k) = VC(G \setminus \{x\}, k - 1) \cup \{x\}$$

1. Justifier les trois règles précédentes.
2. Montrer que si  $G$  possède un vertex cover de taille  $k$  alors le graphe réduit possède au plus  $k^2 + k$  sommets et au plus  $k^2$  arêtes. Pour cela :
  - (a) Montrer que le graphe réduit possède au plus  $k^2$  arêtes.
  - (b) Montrer que le graphe réduit possède au plus  $k^2 + k$  sommets.
3. Montrer que si le graphe réduit possède plus de  $k^2$  arêtes ou plus de  $k^2 + k$  sommets alors  $G$  ne possède pas de solution.

4. Soit  $(x_v)_{v \in V}$  une solution optimale d'une instance  $(G, k)$  du problème de VERTEX COVER obtenue par la relaxation du programme linéaire en nombres entiers et soit  $V_0, V_1, V_{1/2}$  les ensembles des solutions du programme linéaire.

**Solution.**

1. Les trois règles sont clairement cohérentes.
2. Il y a bien au plus  $k^2$  arêtes car toutes les arêtes sont couvertes et on a un vertex cover de taille  $k$ . Alors, chacun des  $k$  sommets du vertex cover a au plus  $k$  voisins (par notre réduction) et donc il y a bien au plus  $k \times k = k^2$  arêtes dans le graphe réduit. De même, on a bien au plus  $k^2 + k$  sommets (de par le degré des sommets dans le graphe).
3. Une solution respecte forcément les critères précédents donc si on ne respecte pas les critères on a un certificat de non existence de solution.

**Exercice 54 Un algorithme de branchement pour le Stable Maximum.**

Proposez un algorithme de branchement pour le problème du STABLE MAXIMUM.

**Solution.** On cherche  $\alpha(G)$  le nombre de stabilité du graphe  $G$  (la taille du plus grand stable dans  $G$ ). On cherche alors

$$\alpha(G) = \max \{ \alpha(V \setminus \{v\}), \alpha(V \setminus N(v)) + 1 \}$$

En effet, soit  $v$  est dans le stable maximum, alors on ne peut pas prendre les voisins de  $v$  et on a un gain de 1 (pour  $v$ ) ou alors  $v$  n'est pas dans le stable maximum et on peut prendre les voisins de  $v$ .

On rappelle que pour  $\Delta \leq 2$  on peut trouver un stable maximum en temps linéaire. Pour  $\Delta \geq 3$  le problème est NP-complet. Notre branchement se fait alors comme suit :

$$\begin{cases} T(n) & \leq T(n-1) + T(n-\Delta-1) \\ & \leq T(n-1) + T(n-4) \end{cases}$$

En se référant à la formule de branchement (dans le cours), on trouve que  $T(n) \leq O(1.3803^n)$ .

**Exercice 55 Un algorithme de branchement pour le 3-SAT.**

Le but de l'exercice est de fournir un algorithme de résolution du problème NP-complet 3-SAT qui fonctionne en temps exponentiel, mais le plus rapide possible.

On précise d'abord quelques notations et définitions. Une instance  $F$  de 3-SAT est une formule booléenne sous forme normale conjonctive telle que chaque clause contienne au plus trois littéraux.

On notera  $n$  le nombre de variables de  $F$  et  $m$  son nombre de clauses. Soit  $c_i = (l_1 \vee l_2 \vee l_3)$  une clause de  $F$ . On définit trois formules obtenues par *assignement partiel* selon  $c_i$ . La formule  $F_1 = F|l_1 = \text{TRUE}$  est obtenue en enlevant à  $F$  toutes les clauses contenant  $l_1$  (car elles sont satisfaites) et en enlevant le littéral  $\neg l_1$  de toutes les clauses contenant  $\neg l_1$  (car elles ne peuvent pas être satisfaites par  $l_1$ ). De même,  $F_2 = F|l_1 = \text{FALSE}, l_2 = \text{TRUE}$  est obtenue en enlevant à  $F$  toutes les clauses contenant  $l_2$  ou  $\neg l_1$  et en enlevant le littéral  $\neg l_2$  ou  $l_1$  de toutes les clauses contenant  $\neg l_2$  ou  $l_1$ . Enfin,  $F_3 = F|l_1 = \text{FALSE}, l_2 = \text{FALSE}, l_3 = \text{TRUE}$  est obtenue en enlevant à  $F$  toutes les clauses contenant  $l_3$  ou  $\neg l_1$  ou  $\neg l_2$  et en enlevant le littéral  $\neg l_3$  ou  $l_1$  ou  $l_2$  de toutes les clauses contenant  $\neg l_3$  ou  $l_1$  ou  $l_2$ .

1. Rappeler un algorithme de résolution de 3-SAT qui fonctionne en temps  $O(m \cdot 2^n)$ .
2. Soit  $F$  une instance de 3-SAT et  $c_i = (l_{i,1} \vee l_{i,2} \vee l_{i,3})$  une clause de  $F$ . Montrer que si une formule obtenue par assignement partiel selon  $c_i$  est vide alors  $F$  est satisfiable. Que dire si une formule obtenue par assignement partiel contient une clause vide ?
3. Montrer que  $F$  est satisfiable si et seulement si l'une des trois formules  $F_1, F_2$  ou  $F_3$  est

satisfiable.

**Solution.**

1. On essaie toutes les permutations de toutes les variables pour chacune des clauses. La complexité est alors de  $O(m \cdot 2^n)$  où  $m$  est le nombre de clauses et  $n$  le nombre de variables.
2. Si une formule obtenue par assignement partiel est vide alors toutes les clauses de  $F$  sont satisfaites, et donc  $F$  est satisfiable. Si une formule obtenue par assignement partiel contient une clause vide alors il existe une clause de  $F$  qui n'est pas satisfaite, et donc  $F$  n'est pas satisfiable.
3. Si  $F$  est satisfiable alors il existe une assignation des variables qui satisfait toutes les clauses de  $F$ . En particulier, il existe une assignation des variables qui satisfait la clause  $c_i$ . Par conséquent, il existe une assignation des variables qui satisfait l'une des trois formules  $F_1$ ,  $F_2$  ou  $F_3$ . Inversement, si l'une des trois formules  $F_1$ ,  $F_2$  ou  $F_3$  est satisfiable alors il existe une assignation des variables qui satisfait toutes les clauses de  $F$ , et donc  $F$  est satisfiable.

## 5 Réductions divers

### 5.1 $L$ -réduction

#### Exercice 56 Exemples de $L$ -réductions de base.

On rappelle les énoncés des deux problèmes suivants :

##### Problème 10 – MAXIMUM $\neq$ 3-SAT

**Input:** Même données que pour 3-SAT

**Question:** Trouver le maximum de clauses satisfaites telle que dans chaque clause satisfaite il existe au moins un littéral mis à vrai et au moins un littéral mis à faux.

##### Problème 11 – MAXIMUM MULTI-COUPÉ (MAXIMUM MULTI-CUT)

**Input:** Un multigraphe  $G = (V, E)$  et un entier  $k$

**Question:** Trouver un ensemble de sommets  $V_1, V_2 \subseteq V$  tel que  $V_1 \cup V_2 = V$  et que le nombre d'arêtes entre  $V_1$  et  $V_2$  est au moins  $k$ .

Faire les  $L$ -réductions sur les problèmes suivants :

1. 2-SATISFIABILITE MAXIMALE  $\leq_L$  MAXIMUM  $\neq$  3-SAT
2. MAXIMUM  $\neq$  3-SAT  $\leq_L$  MAXIMUM MULTI-COUPÉ. On propose la construction suivante :

**Construction 2.** Considérons une instance  $\varphi$  de MAXIMUM  $\neq$  3-SAT sur  $n$  variables et  $m$  clauses et supposons que chacune des  $m$  clauses de  $\varphi$  contient au moins un littéral.

- Pour toute variable  $x$  de  $\varphi$ , le graphe  $G = f(\varphi)$  contient deux sommets  $v_x$  et  $v_{\bar{x}}$ ; ces sommets sont reliés entre eux par  $2n_x$  arêtes parallèles, où  $n_x$  est le nombre de fois que la variable  $x$  apparaît dans  $\varphi$ . Nous appelons ces arêtes “ $v$ -arêtes” pour indiquer qu’elles sont associées aux variables de  $\varphi$ .
- Pour chaque clause  $C = (y \vee z \vee w)$  de  $\varphi$ , le graphe  $G$  contient un triangle sur les sommets  $v_y, v_z$  et  $v_w$ ; pour chaque clause  $C = (y \vee z)$  de  $\varphi$ , le graphe  $G$  contient deux arêtes parallèles entre les sommets  $v_y$  et  $v_z$ . Nous allons appeler ces arêtes “ $c$ -arêtes” pour indiquer qu’elles sont associées aux clauses de  $\varphi$ . La fonction  $f$  de la réduction est ainsi spécifiée. Il est facile de voir que le graphe ainsi construit est un multigraphe, instance de MAXIMUM MULTI-COUPÉ.

#### Solution.

1. On considère la transformation suivante :

$$(l_1^i, l_2^i) \mapsto (l_1^i, l_2^i, x)$$

Alors, toute solution de 2-SATISFIABILITE MAXIMALE est aussi une solution de MAXIMUM  $\neq$  3-SAT, il suffit de mettre  $x$  à faux. De plus, toute solution de MAXIMUM  $\neq$  3-SAT est aussi une solution de 2-SATISFIABILITE MAXIMALE, il suffit d’ignorer les clauses contenant  $x$ . Par conséquent, on a bien une  $L$ -réduction avec  $\alpha_1 = \alpha_2 = 1$ .

On se propose dans la suite de montrer les relations suivantes :

$$\min \text{VC}(3) \prec_L \min \text{DOM}(6) \prec_L \min \text{DOM}(3)$$

Montrons d’abord que  $\min \text{VC}(3) \prec_L \min \text{DOM}(6)$ . Soit  $G$  un graphe. On propose la construction suivante :

$$\forall (u, v) \in E(G),$$

On construit le triangle  $(u, v, w)$ .

Montrons maintenant qu’on a  $\forall k, \text{VC}(k) \approx \text{DOM}(k)$ .

Si on a une solution à  $VC(k)$ , le même ensemble de sommets est bien une solution à  $DOM(k)$  puisque tout sommet de  $G$  est soit dans la solution, soit adjacent à un sommet de la solution. Si on a une solution à  $DOM(k)$ , on effectue les opérations suivantes :

- Si  $u$  ou  $v$  est dans la solution, on ajoute  $u$  et  $v$  à la solution.
- Sinon,  $w$  est dans la solution, on ajoute alors  $u$  ou  $v$  à la solution.

On a donc égalité des solutions optimales pour les deux problèmes, et on a aussi égalité des solutions approximatives, ce qui montre que  $VC(k) \approx DOM(k)$  avec  $\alpha = \beta = 1$ .

Si on regarde maintenant le degré des graphes, dans un graphe subcubique, un sommet transformé en triangle a un degré au plus égal à 6, d'où la réduction de  $VC(3)$  à  $DOM(6)$ .

Montrons maintenant que  $\min DOM(6) \prec_L \min DOM(3)$ .

### Exercice 57 Exemples de $L$ -réductions sur les graphes.

1. Montrer que MINIMUM VERTEX COVER-3 est  $\mathcal{APX}$ -complet. Utiliser la transformation proposée par la figure 9 pour vous aider. La réduction se fera à partir du problème MINIMUM VERTEX COVER-4. Conclure que MAXIMUM INDEPENDENT SET-3 est aussi  $\mathcal{APX}$ -complet.

### Solution.

1. On utilise la transformation  $f$  décrite dans la construction ci-dessus. Soit  $s$  le nombre de sommets de degré 4. Soit  $k$  la taille d'un vertex cover de  $G$ . Alors, la transformation de  $G$  par  $f$  donne un vertex cover solution de taille  $k' = k + s$ . On a donc

$$|C'| = |C| + s$$

On sait que  $|E| \geq |V|$ . De plus,

$$\sum_{v \in V} \deg(v) = 2|E| \geq |E|$$

et on a aussi

$$4|C| \geq \sum_{v \in C} \deg(v) \geq |E| \geq |V| \geq s$$

Par conséquent, on a

$$\begin{aligned} \text{OPT}(G') &= \text{OPT}(G) + s \\ &= 5 \cdot \text{OPT}(G) \\ \implies \alpha_1 &= 5 \end{aligned}$$

On a aussi

$$|C| \leq |C'| - s$$

pour tout vertex cover. En particulier,

$$\text{OPT}(G) \leq \text{OPT}(G') - s$$

et

$$\begin{aligned} \text{OPT}(G) - |C| &\leq \text{OPT}(G') - s - (|C'| - s) \\ &= \text{OPT}(G') - |C'| \\ \implies \alpha_2 &= 1 \end{aligned}$$

Par conséquent, on a bien une  $L$ -réduction avec  $\alpha_1 = 5$  et  $\alpha_2 = 1$ .

## 6 Quelques problèmes d'ordonnancement

### 6.1 Ordonnancement sur un processeur

**Exercice 58** title.

**Solution.**

### 6.2 Ordonnancement sur $m$ processeurs

**Exercice 59 Problème d'ordonnancement sans contrainte de précédence.**

On pose  $P| \cdot |C_{\max}$  définie de la manière suivante :

**Problème 12** –  $P| \cdot |C_{\max}$

**Input:**  $n$  tâches de durées  $p_1, \dots, p_n$  indépendantes

**Question:** Ordonnancer ces tâches sur  $m$  machines identiques en une durée totale minimale notée  $C_{\max}$ .

Par la suite on va utiliser un ordonnancement par liste. Le principe est le suivant :

- On construit une liste de priorité de tâches.
- A chaque étape la première machine disponible est sélectionnée pour exécuter la première tâche de la liste.

1. Montrer que le problème est  $\mathcal{NP}$ -complet même pour deux machines.

2. Pour n'importe quel ordonnancement de liste  $LS$  on a  $\frac{C_{\max}^{LS}}{C_{\max}^*} \leq 2$ . Montrer que la borne est supérieure est atteinte.

3. Nous considérons le nouvel algorithme dont le principe est le suivant :

- On construit une liste de priorité de tâches dans l'ordre décroissant.
- A chaque étape la première machine disponible est sélectionnée pour exécuter la première tâche de la liste.

Pour n'importe quel ordonnancement de liste on a  $\frac{C_{\max}^{LPT}}{C_{\max}^*} \leq \frac{4}{3} - \frac{1}{3m}$ .

4. Un schéma d'approximation polynomiale pour le problème  $P| \cdot |C_{\max}$

**Solution.**

1. On va faire une réduction du problème de 2-PARTITION vers  $P| \cdot |C_{\max}$ . On rappelle l'énoncé du problème 2-PARTITION :

**Problème 13** – 2-PARTITION

**Input:** Un ensemble de  $n$  objets  $a_i$  de poids  $s(a_i)$  de somme  $2p$

**Question:** Est-il possible de trouver une partition en deux sous-ensemble de poids total  $p$

Dans le problème de 2-PARTITION, on a

$$\sum_{i=1}^n s(a_i) = 2p$$

On pose

$$s(a_i) = p_i \quad \text{et} \quad w = \sum_{i=1}^n p_i$$

Montrons que si il est possible de partager les poids en deux sous-ensembles de même poids alors il existera une distribution des tâches pour le problème  $P \mid \cdot \mid C_{\max}$  telle que

$$C_{\max} \leq T \leq \frac{w}{2}$$

Si on peut partager les poids en deux sous-ensembles  $P_1, P_2$  alors il existe  $S$  tel que

$$\text{Load}(M_1) = \sum_{i \in S} p_i = \sum_{a_i \in P_1} s(a_i) = p$$

de même pour l'autre machine. Alors on a bien trouvé une solution.

Pour le sens indirect, soit  $C$  une distribution telle que

$$C_{\max} \leq \frac{w}{2}$$

Soit  $S$  un sous-ensemble de travaux de sorte que  $S \subseteq \{1, \dots, n\}$  et

$$\text{Load}(M_1) = \sum_{i \in S} p_i = \frac{w}{2}$$

Alors, comme pour tout  $i$  on a  $p_i = s(a_i)$ , l'ensemble des tâches sur le processeur  $P_1$  forme une solution du problème de  $P \mid \cdot \mid C_{\max}$  avec une bipartition

$$\sum_{i \in P_1} s(a_i) = p$$

Les éléments restants sont dans  $P_2$  ce qui donne une solution à 2-PARTITION.

2. On peut déjà trouver deux bornes inférieures intéressantes :

$$C_{\max}^{OPT} \geq \max_{i \in I} p_i, \quad \text{et} \quad C_{\max}^{OPT} \geq \frac{1}{m} \sum_{i=1}^n p_i$$

Il faut au moins la durée de la tâche de la plus longue et on ne peut pas faire mieux que la répartition parfaite de toutes les tâches sur tous les processeurs de manière continue.

On a

$$C_{\max} = t_j + p_j \leq \max_i p_i + t_j \leq C_{\max}^{OPT} + t_j$$

Mais de par la nature de notre algorithme, avant  $t_j$  tous les processeurs sont actifs d'où

$$C_{\max} = t_j + p_j \leq 2C_{\max}^{OPT}$$

Autre correction possible :

On a

$$C_{\max}^{OPT} \geq \frac{T_{\text{seq}}}{m}$$

et

$$C_{\max}^{OPT} \geq p_j, \quad \forall j$$

Si on considère l'agencement des tâches on trouve

$$T_{\text{seq}} \geq (C_{\max} - p_j) m + p_j$$

et alors

$$\frac{T_{\text{seq}}}{m} \geq C_{\max} - p_j + \frac{p_j}{m}$$



en combinant ces deux inégalités on trouve

$$\begin{aligned} C_{\max}^{LS} &\leq \frac{T_{\text{seq}}}{m} + p_j - \frac{p_j}{m} \\ &\leq \frac{T_{\text{seq}}}{m} + p_j \left(1 - \frac{1}{m}\right) \\ &\leq C_{\max}^{OPT} + C_{\max}^{OPT} \left(1 - \frac{1}{m}\right) \\ &\leq C_{\max}^{OPT} \left(2 - \frac{1}{m}\right) \\ &\leq 2C_{\max}^{OPT} \end{aligned}$$

**Exercice 60 title.**

**Solution.**

**Exercice 61 title.**

**Solution.**

**Exercice 62 title.**

**Solution.**

## 7 Algorithmes randomisés

### 7.1 Sur le problème Maximum Satisfaisabilité

Exercice 63 title.

Solution.

### 7.2 Sur le problème Couverture d'ensembles pondérés

Exercice 64 title.

Solution.

Exercice 65 title.

Solution.

Exercice 66 title.

Solution.

Exercice 67 title.

Solution.

Exercice 68 title.

Solution.

## 8 Bornes inférieures et ETH

Exercice 69 title.

Solution.

Exercice 70 title.

Solution.

Exercice 71 title.

Solution.